

EEE3096S Practical 1B

Phila Haanyama
HNYPHI001

Lulama Lingela
LNGLUL002

I. INTRODUCTION

In this practical, the STM32F0 microcontroller was tested to see how well it could handle running a computationally heavy task by using a Mandelbrot set algorithm. The Mandelbrot set is a type of mathematical fractal defined in the complex plane and is generated via iterative calculations that can be computationally extensive. By implementing this set algorithm on the STM32F0, this practical allowed for the measurement of execution time and calculation of a checksum which provided a means of testing the microcontroller's processing speed.

Two versions of the algorithm was written: one using fixed-point arithmetic (only integers, with a scaling factor to represent decimals) and one using double-precision floating-point numbers. How long each version took to run for different image sizes was then measured and compared to reference values generated in Python. This gave a clear picture of the trade-offs between using fixed-point and floating-point maths on a low-powered microcontroller.

II. METHODOLOGY

1) Setting up the project

Cloned the provided GitHub repository and imported the *Practical_1B* project into STM32CubeIDE.

2) Adding global variables

Declared global variables to store execution timing, checksum results, and the set of image dimensions to test (128, 160, 192, 224, 256).

3) Fixed-point implementation

Implemented `calculate_mandelbrot_fixed_point_arithmetic()` using only integers and a scale factor of 10^6 to represent decimal values.

Used 64-bit integers for intermediate calculations to prevent overflow. Utilising fixed-point arithmetic instead of just using integers is necessary in order to maintain all fractional precision in coordinate mapping. The boundaries of the Mandelbrot set require fractional resolution to determine the iteration counts of escapes accurately.

4) Timing and debugging

Used `HAL_GetTick()` to record start and end times around the Mandelbrot function, calculating execution time in milliseconds.

Monitored the `execution_time` and `checksum` variables for each image size in the STM32CubeIDE Live Expressions window.

5) Double-precision algorithm

Implemented `calculate_mandelbrot`

`_double()` using double-precision variables for the same algorithm.

Measured execution time and checksum again for all image sizes.

6) Comparing with Python

Ran the provided Python script to obtain reference checksum values.

Compared the Python results with the STM32 outputs for both algorithms.

7) Recording results

Tabulated all timing and checksum values.

Noted any differences between the fixed-point and double-precision algorithms.

III. RESULTS & DISCUSSION

Image Size	Time (s)	Checksum
128×128	20.917	430527
160×160	32.648	671374
192×192	47.025	967630
224×224	64.038	1317555
256×256	83.549	1718118

TABLE I

FIXED-POINT EXECUTION TIMES AND CHECKSUMS.

Image Size	Time (s)	Checksum
128×128	121.167	429384
160×160	190.458	669829
192×192	274.594	966024
224×224	374.223	1314999
256×256	485.450	1715812

TABLE II

DOUBLE-PRECISION EXECUTION TIMES AND CHECKSUMS.

Image Size	Time (s)	Checksum
128×128	0.032	429384
160×160	0.072	669829
192×192	0.071	966024
224×224	0.095	1314999
256×256	0.123	1715812

TABLE III

PYTHON EXECUTION TIMES AND CHECKSUMS (REFERENCE).

The execution time for the Mandelbrot methods increased as the image size increased. When comparing the two different implementations on the STM32F0, using double-precision took consistently approximately 5–6× longer than fixed-point (Figure 1). This delay was expected, as the STM32F0 does

not have a floating-point unit (FPU) and therefore emulates floating-point operations in software, whereas fixed-point arithmetic can utilise native integer operations.

There is a large performance gap between the microcontroller and the laptop (M4 MacBook Pro) processor. This is observed when completing the largest image: STM32F0 required 83.5 s (fixed-point) and 485.5 s (double), while the PC took only 0.12 s (Tables I, II, and III). This difference arises due to factors such as clock speed, memory bandwidth, and the presence of hardware FPUs on the PC.

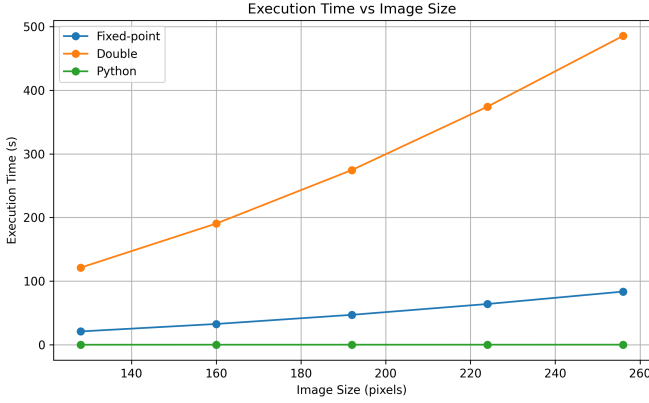


Fig. 1. Execution time vs image size for Fixed-point, Double, and Python implementations.

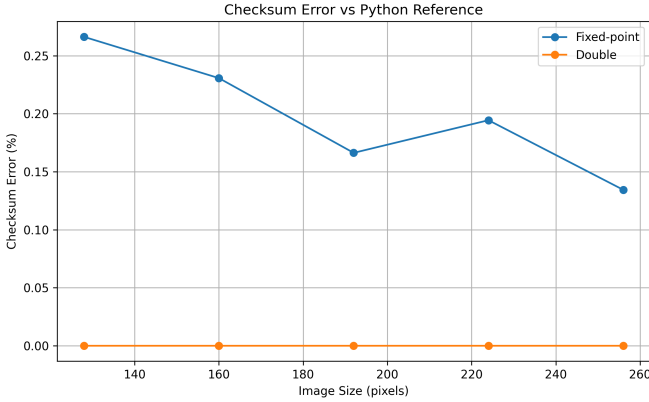


Fig. 2. Percentage checksum error vs image size compared to Python reference.

The checksum value was calculated as the total number of iteration counts for all the pixels in the generated Mandelbrot set image. It acts as a fingerprint of the final output - if two runs give the same checksum, it means the images are identical.

Using double-precision produced identical checksum results to the Python reference for all image sizes, whereas fixed-point deviated by at most 0.27%. This is due to rounding and truncation errors that occur during fixed-point coordinate mapping and iterations. However, the effect of these differences in individual pixel iteration counts is minimal, as the fixed-point

Image Size	Fixed-point Error (%)	Double Error (%)
128×128	0.2663	0.0000
160×160	0.2309	0.0000
192×192	0.1661	0.0000
224×224	0.1940	0.0000
256×256	0.1342	0.0000

TABLE IV
PERCENTAGE ERROR IN CHECKSUM COMPARED TO PYTHON REFERENCE.

implementation has a tolerance within $\pm 1\%$ as specified in the brief.

The checksums from the double-precision implementation are more accurate than the fixed-point results. However, there is a trade-off: the fixed-point method runs much faster on the STM32F0, as it does not have a floating-point unit (FPU), which makes integer operations much faster than emulating floating-point operations. Utilising single-precision floats (32-bit) instead of doubles (64-bit) may run slightly faster than the current double-precision method, but will be less accurate due to increased rounding errors.

IV. CONCLUSION

This practical involved comparing the performance and accuracy of fixed-point and double-precision Mandelbrot implementations on the STM32F0 microcontroller, and comparing the results with a Python script reference running on a PC. The benchmark results showed that fixed-point arithmetic has an approximately 5–6 $\times$ faster execution time on the STM32F0 in comparison to double-precision, which is slower due to the lack of a hardware FPU. However, double-precision produces accurate results that align exactly with the Python reference output, while fixed-point has a minimal accuracy loss (within $\pm 1\%$ tolerance). The choice between the two depends on whether execution speed or exact numerical accuracy is more important for the application.

For future improvements, using an STM32F4 would enhance the performance of running the Mandelbrot algorithm compared to using an STM32F0. The STM32F4 includes an FPU and has a higher clock speed, which would optimise the execution time for both the fixed-point and double-precision implementations.

V. AI CLAUSE

- Gaining a better understanding about the task through high-level explanations.
- Getting detailed explanations of errors during coding.
- Generated Python code for graphing recorded execution time and checksum results.
- Used to debug $\LaTeX$ formatting issues.
- Improving clarity & readability of the report.

Appendix

Github repo: <https://github.com/LlamaLuLu/EEE3096S-Practicals-Group-34.git>

EEE3096S: Practical 1B Code

Phila Haanyama (HNYPHI001) and Lulama Lingela (LNLUL002)

File: Practical_1B/Core/Src/main.c

```
/* USER CODE BEGIN Header */
/**
  *****************************************************************************
  * @file      : main.c
  * @brief     : Main program body
  *****************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *****************************************************************************
  */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdint.h>
#include "stm32f0xx.h"
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */
#define MAX_ITER 100
/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */
/* USER CODE END PM */

/* Private variables -----*/
/* USER CODE BEGIN PV */
//TODO: Define and initialise the global variables required
/*
  start_time
  end_time
  execution_time
  checksum: should be uint64_t
  initial width and height maybe or you might opt for an array??
  */
uint32_t start_time = 0;
```

```

uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;

int32_t height = 0;
int32_t width = 0;

const int image_dimensions[] = {128, 160, 192, 224, 256};

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int
max_iterations);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick.
    */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();

    /* USER CODE BEGIN 2 */

    for (int i = 0; i < 5; i++) {
        width = image_dimensions[i];
        height = image_dimensions[i];

        //TODO: Turn on LED 0 to signify the start of the operation
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
    }
}

```

```

        // Fixed-point benchmark:
        //TODO: Record the start time
        start_time = HAL_GetTick();
        //TODO: Call the Mandelbrot Function and store the output in the checksum
variable defined initially
        checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height,
MAX_ITER);
        //TODO: Record the end time
        end_time = HAL_GetTick();
        //TODO: Calculate the execution time
        execution_time = end_time - start_time;
        // Store or print execution_time & checksum

        // Double benchmark:
        //TODO: Record the start time
        start_time = HAL_GetTick();
        //TODO: Call the Mandelbrot Function and store the output in the checksum
variable defined initially
        checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
        //TODO: Record the end time
        end_time = HAL_GetTick();
        //TODO: Calculate the execution time
        execution_time = end_time - start_time;
        // Store or print execution_time & checksum

        //TODO: Turn on LED 1 to signify the end of the operation
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);

        //TODO: Hold the LEDs on for a 1s delay
        HAL_Delay(1000);

        //TODO: Turn off the LEDs
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
    }

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Initializes the RCC Oscillators according to the specified parameters
     * in the RCC_OscInitTypeDef structure.
     */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {

```

```

    Error_Handler();
}

/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                             |RCC_CLOCKTYPE_PCLK1;
RCC_ClkInitStruct.SYSClockSource = RCC_SYSCLOCKSOURCE_HSI;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLOCK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */
    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOB_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1, GPIO_PIN_RESET);

    /*Configure GPIO pins : PB0 PB1 */
    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */
    /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

//TOD0: Mandelbrot using variable type integers and fixed point arithmetic
#define SCALE 4096 // scaling factor
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int
max_iterations){
    uint64_t mandelbrot_sum = 0;
    //TOD0: Complete the function implementation
    for(int y = 0; y < height; y++){
        for(int x = 0; x < width; x++){
            // convert pixel coordinates to fixed point complex plane
coordinates
            int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; //
making sure all numbers are integers not floats
            int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

            int32_t xi = 0;
            int32_t yi = 0;
            int i = 0;

```

```

        while (i < max_iterations){
            int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
            int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

            if(xi_sq + yi_sq <= 4 * SCALE){
                int32_t temp = (int32_t)(xi_sq - yi_sq);

                int64_t xy = ((int64_t)xi * yi) / SCALE;

                yi = (int32_t)(2*xy + y0);
                xi = temp + x0;

                i++;
            }
            else{
                break;
            }
        }

        mandelbrot_sum += i;
    }
}
return mandelbrot_sum;
}

//TODO: Mandelbroat using variable type double
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
    uint64_t mandelbrot_sum = 0;
    //TODO: Complete the function implementation
    for (int y = 0; y < height; ++y) {
        double y0 = ((double)y / (double)height) * 2.0 - 1.0;    // map y
        -> [-1, +1]
        for (int x = 0; x < width; ++x) {
            double x0 = ((double)x / (double)width) * 3.5 - 2.5;    // map x
            -> [-2.5, 1.0]
            double xi = 0.0;
            double yi = 0.0;
            int i = 0;

            while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                double tmp = xi*xi - yi*yi + x0;
                yi = 2.0 * xi * yi + y0;
                xi = tmp;
                ++i;
            }
            mandelbrot_sum += (uint64_t)i;
        }
    }

    return mandelbrot_sum;
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */

```

```

    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 *        where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line
    number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```